PROJECT: OSBOT_FAST_API_SERVERLESS

Services Registry Architecture

Implementation Blueprint & Technical Specification

VERSION:	v1.33.0
STATUS:	READY FOR IMPLEMENTATION
DATE:	JANUARY 2025
DOC TYPE:	TECHNICAL BRIEF

Executive Summary: The Capabilities

The Services Registry is a centralized, service-agnostic mechanism for registering and discovering FastAPI service clients. It unifies IN_MEMORY and REMOTE workflows under a single architectural contract.

01

Zero Code Changes

Business logic remains identical whether running locally or distributed. No conditional logic in consumer code.

02

Explicit Registration

Clients must be registered at startup. No "magic" auto-discovery ensures predictable runtime behavior.

03

Type-Safe Retrieval

Clients are indexed by their class type, ensuring full IDE support, auto-completion, and static analysis.

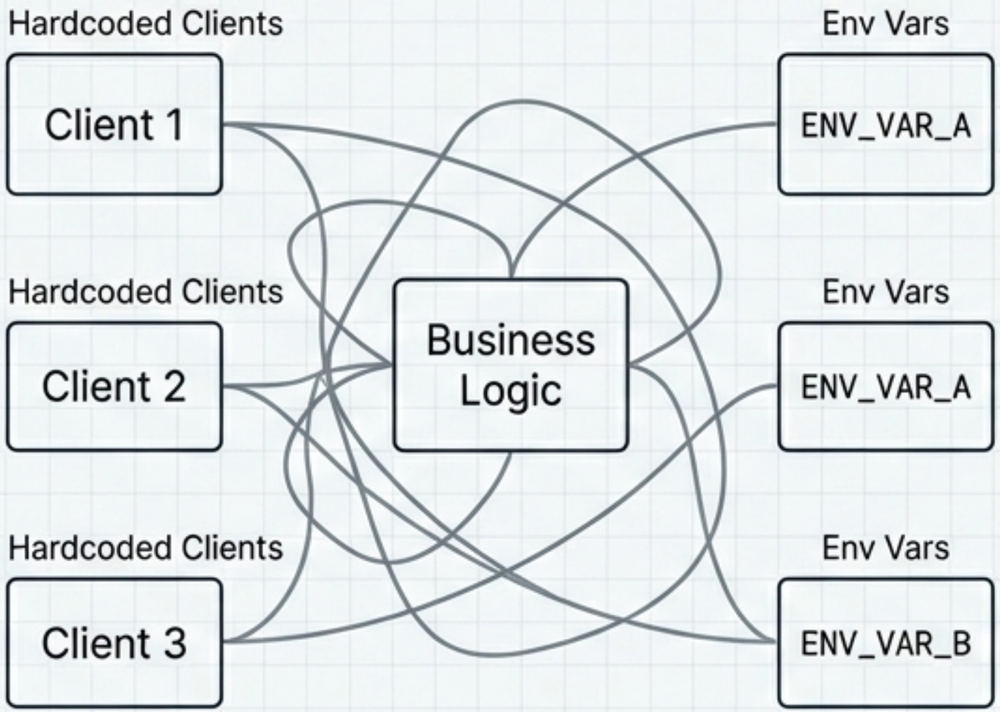
04

Test Isolation

A dedicated `clear()` method ensures clean state between unit tests, preventing side-effect pollution.

Architectural Shift

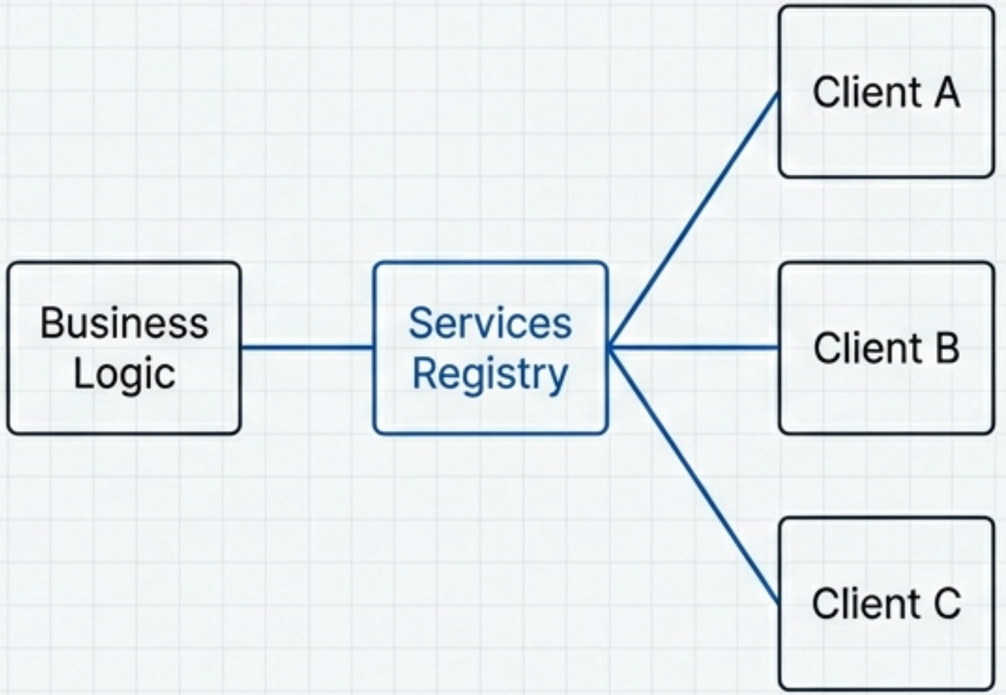
Before: Fragmented Discovery



High Coupling / Hard to Test

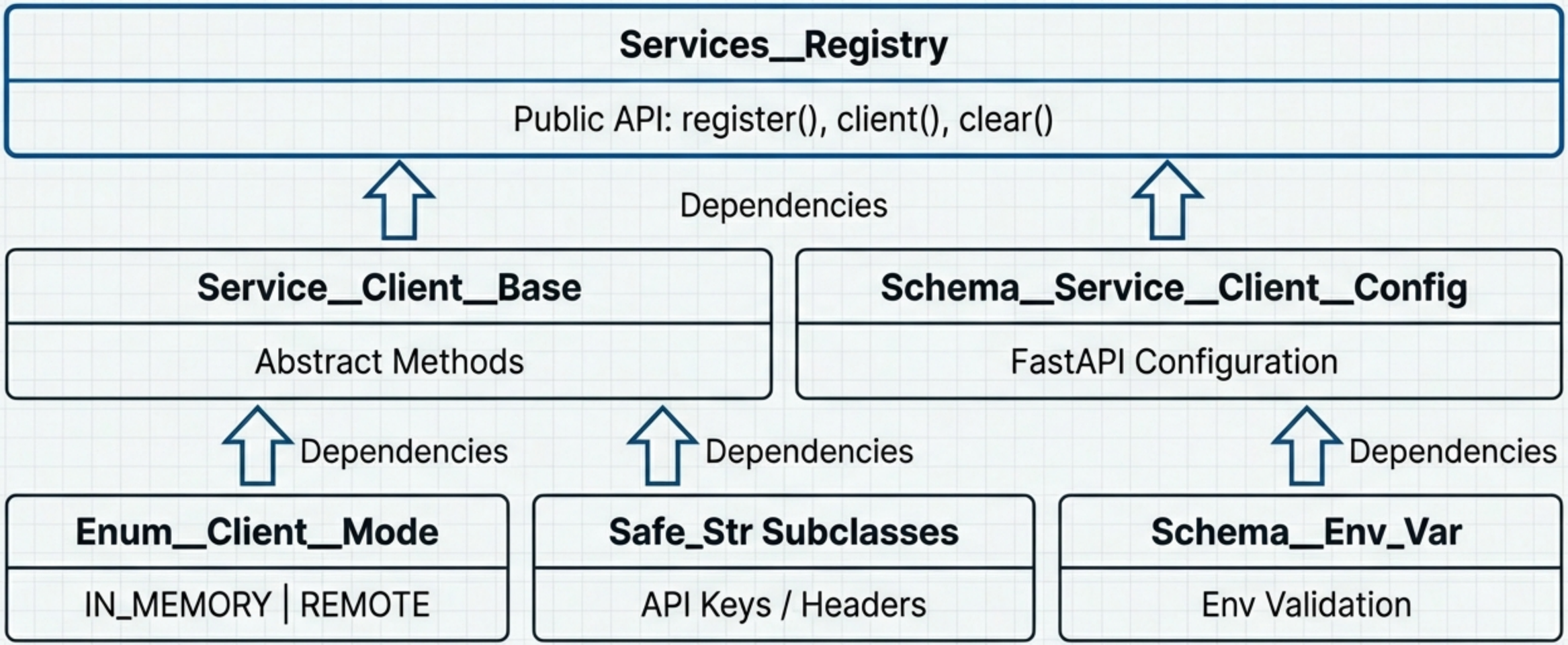
Abstraction Layer

After: Centralized Abstraction



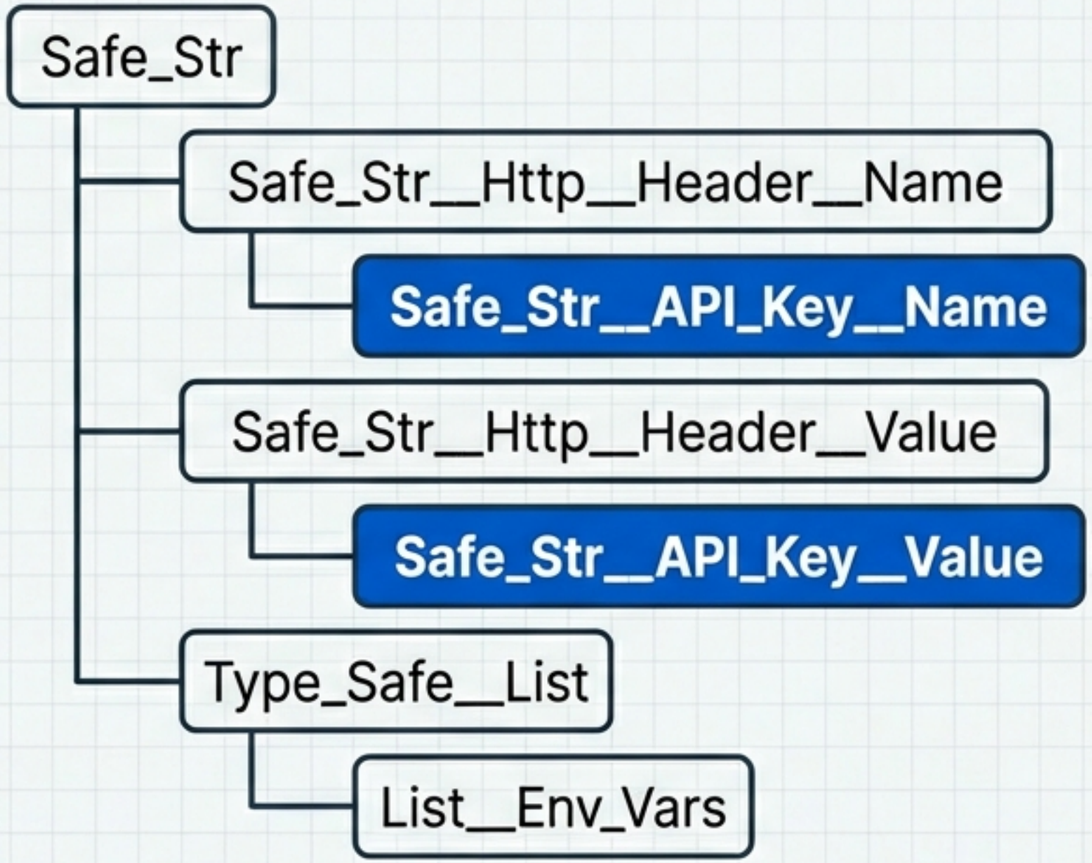
Decoupled / Configurable

Core Implementation Architecture



Safe Primitives & Configuration

Inheritance Chain



Configuration Schema

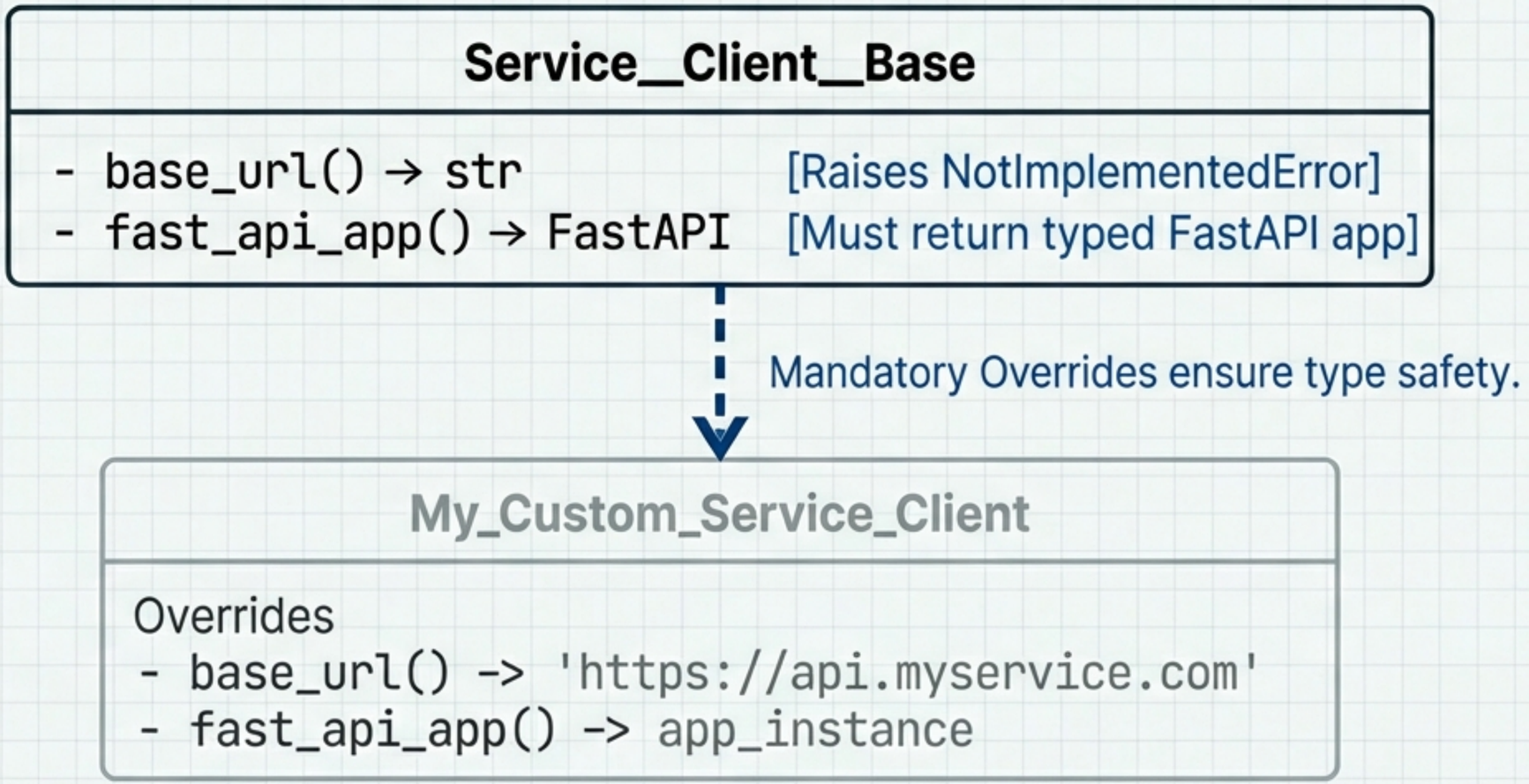
Schema__Env_Var

```
- name: str
- required: bool
```

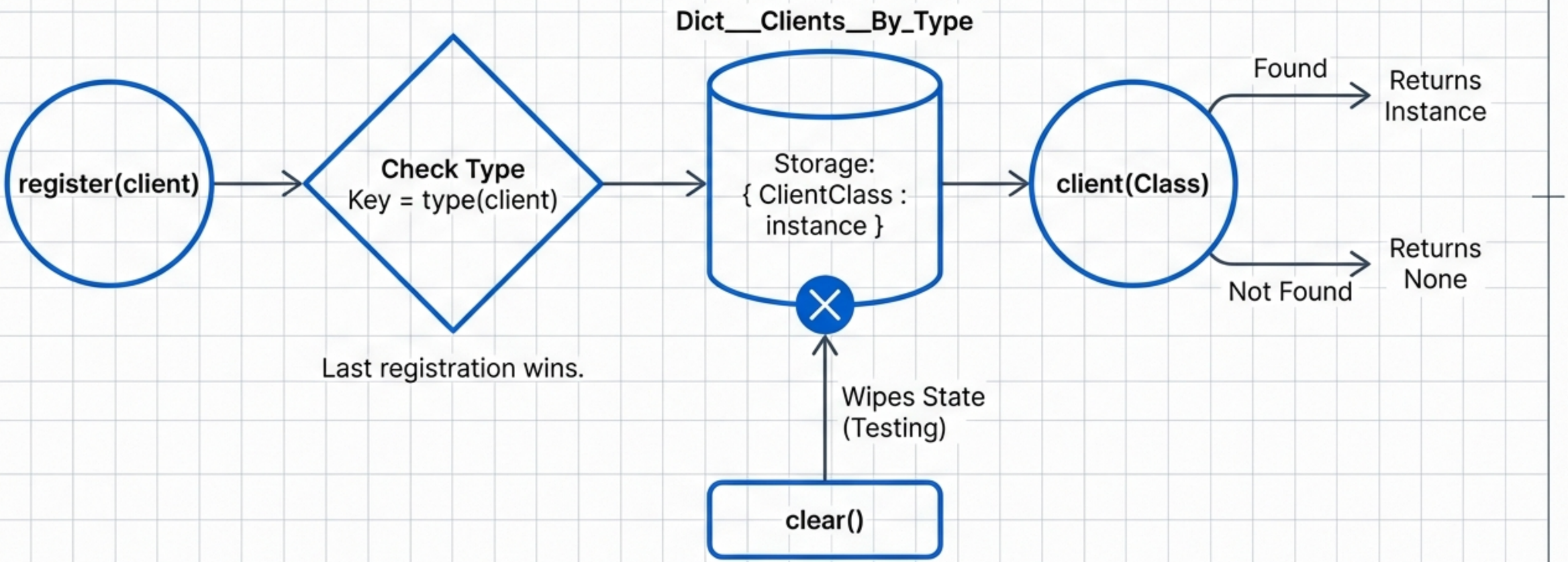
Enforces domain-specific validation and prevents raw string errors.

The Base Class Contract

Enforcing consistency across all services.

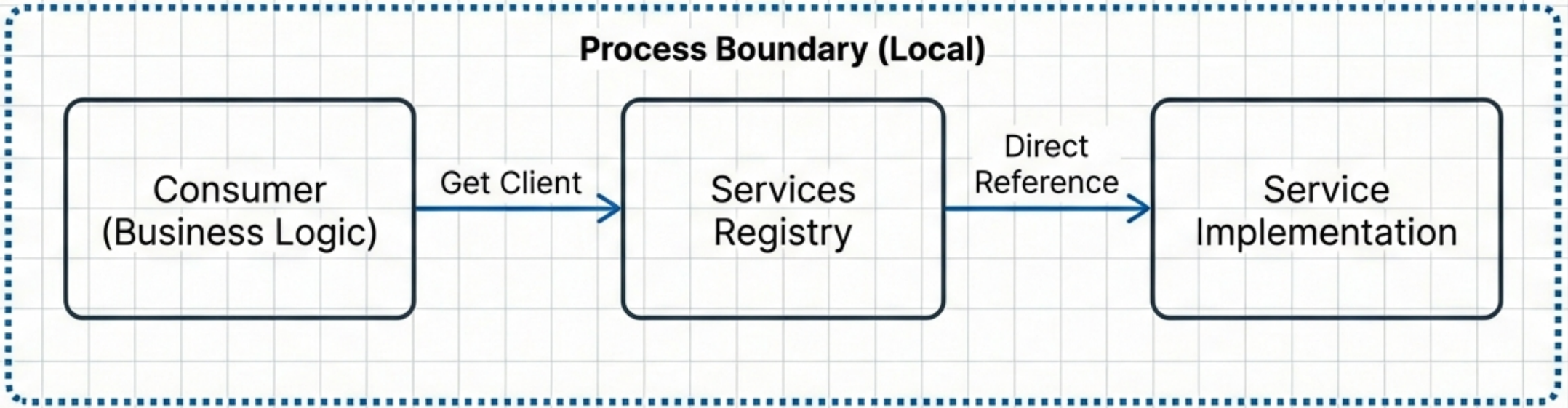


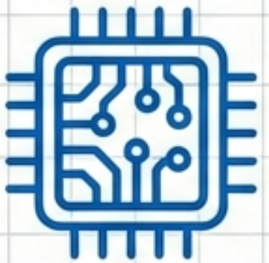
Registry Logic & State Management



Deployment Pattern A: Full Standalone

IN_MEMORY Configuration

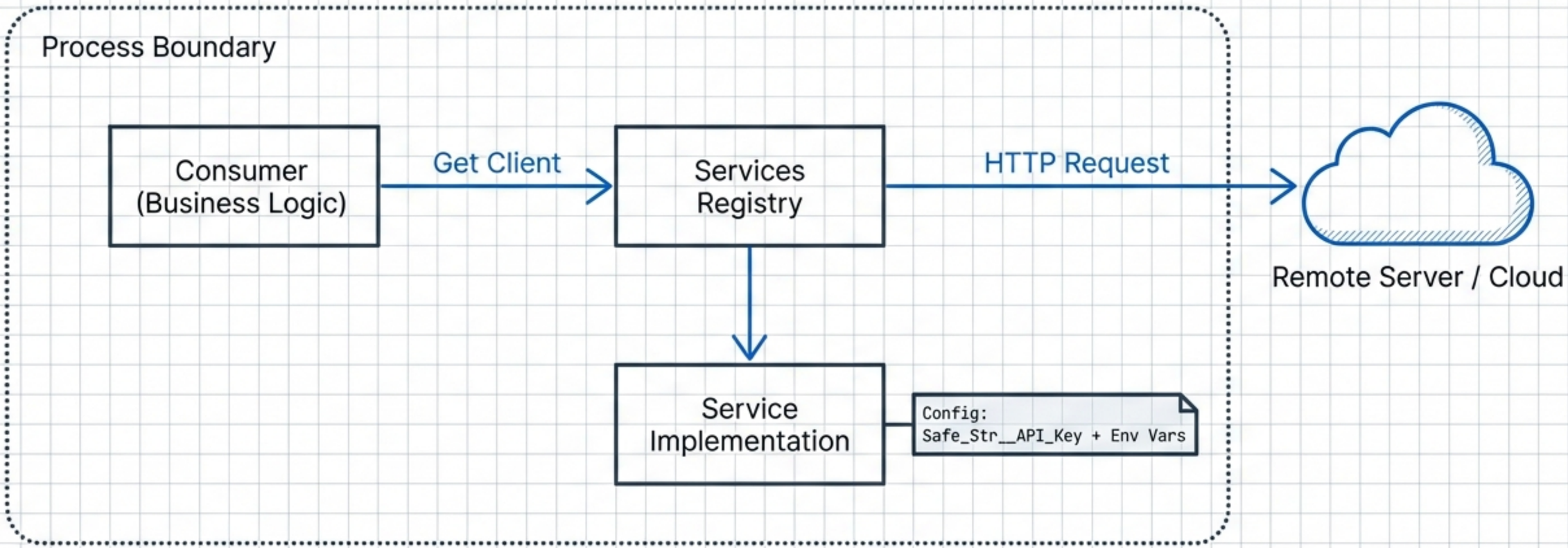




Air-Gapped / Testing Safe. No Network Calls.

Deployment Pattern B: Production

REMOTE Configuration



Mandatory Integration with Auth & Environment

Unified Consumption Pattern

The Developer Experience

```python
from osbot_fast_api_serverless.utils.Services__Registry import
     Services__Registry
from my_services import My_Service_Client

def my_business_logic():
    # Type-safe retrieval
    client = Services__Registry.client(My_Service_Client)

    if client:
        return client.get_data()
    else:
        handle_missing_service()
```

Indexed by Class Type (Not String)

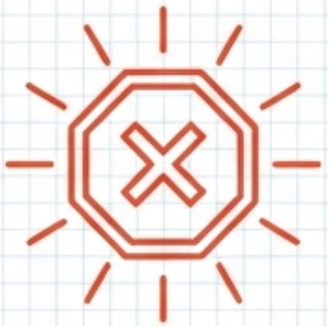
Graceful handling of 'None'

IDE Autocomplete Enabled

Design Decisions & Rationale

Decision	Choice	Rationale
Key Type	Python `Class (type)`	Natural indexing, refactoring support, and IDE intelligence.
Unregistered Behavior	Return `None`	Explicit is better than implicit. Caller decides flow.
Env Var Fallback	No Auto-Creation	Prevents ‘magic’ values. Forces explicit config.
Test Isolation	method: `clear()`	Essential for independent, side-effect-free unit tests.
API Key Types	`Safe_Str` Subclasses	Prevents passing raw strings to sensitive fields.

Error Handling Strategy



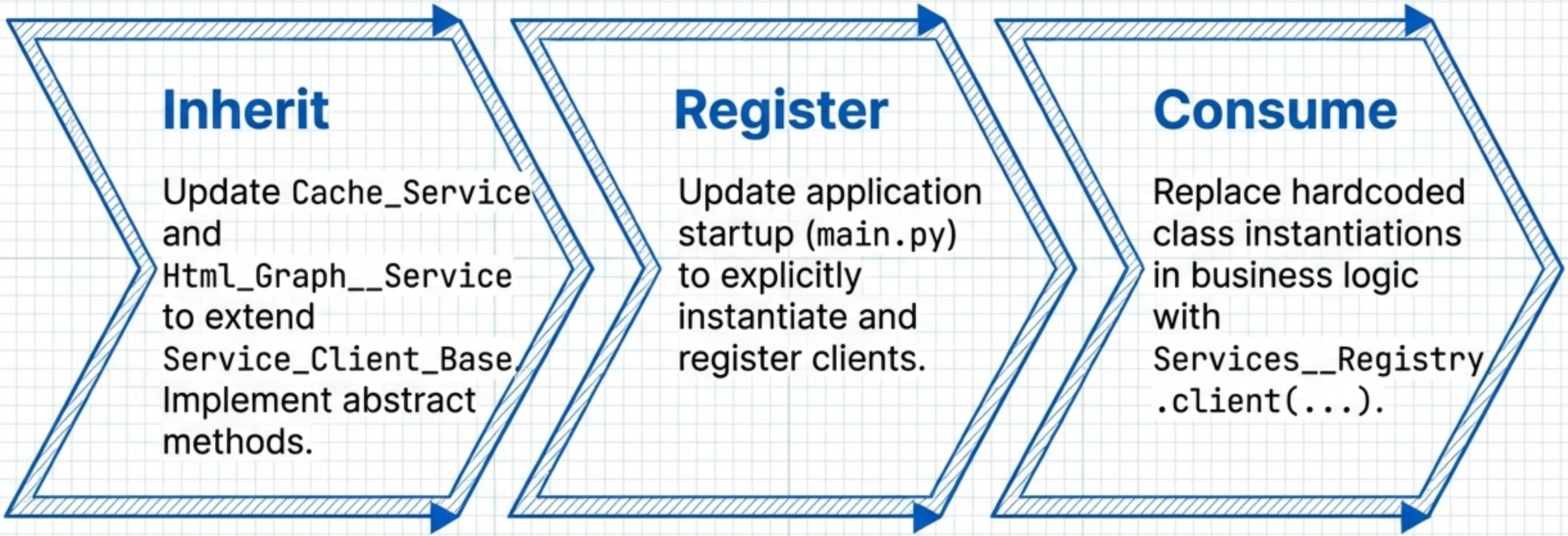
Blocking Errors (Registration)

- `ValueError: Cannot register None`
  - (Cause: Instantiation failure).
- `TypeError: Must inherit from Service__Client__Base`
  - (Cause: Invalid contract).

Non-Blocking Errors (Discovery)

- Client Not Registered
  - Behavior: Returns `None`.
  - Action: Caller must handle or raise specific error.
- Duplicate Registration
  - Behavior: Last registration wins.
  - Action: Silent overwrite (Log warning optional).

Migration Guide



Target: Minimal changes to existing business logic.

Implementation Checklist

Component Creation

- [ ] Create `Enum__Client__Mode` & `Safe_Str` subclasses
- [ ] Create `Schema__Env_Var` & `List__Env_Vars`
- [ ] Create `Schema__Service__Client__Config`
- [ ] Create `Service__Client__Base` (Abstract)
- [ ] Create `Dict__Clients__By_Type` & `Services__Registry`

System Integration

- [ ] Update `Cache__Service__Fast_API__Client`
- [ ] Update `Html_Graph__Service__Client`
- [ ] Refactor Test Fixtures to use `clear()`
- [ ] Verify 'Last Wins' registration logic
- [ ] Document startup patterns for Remote/Local

References & Documentation

Type_Safe Guide

v3_63_4__for_llms__type_safe.md

Collections Guide

v3_63_3__for_llms__type_safe__collections.md

Architecture Spec

22_Jan_-_Standalone_System_Architecture.pdf

Deployment Modes

23_Jan-_Deployment_Modes_Scaling.pdf

Adherence to these standards ensures system-wide consistency.